

Evaluating Supervised Regression Models for Predicting Total Power Output in Wave Energy Farms

Justin Hovious

CSCI 460

May 4, 2026

Abstract

Understanding how different machine learning models perform on structured regression tasks is essential for selecting reliable prediction systems. This project evaluates six regression models: Linear Regression, Ridge, Lasso, Random Forest, Extra Trees, and XGBoost to determine which approach most effectively models the relationship between the input features and power output. Prior to modeling, the dataset was cleaned and simplified by removing redundant or unhelpful variables, ensuring each model was trained on a consistent set of features.

To test and evaluate the models, a consistent preprocessing and hyperparameter-tuning pipeline was constructed for all models, using standardized inputs and a cross-validated grid search. Each model was then evaluated on both the training and test sets using mean absolute error (MAE), mean square error (MSE), root mean square error (RMSE), and the coefficient of determination (R^2). Visualizations, including bar charts, tables, and residual analyses, were used to compare performance and describe model behavior.

The results demonstrate the contrast between linear and tree-based methods. Linear models achieved near-perfect predictive accuracy, while nonlinear models displayed larger errors and clear patterns in the residual plots. These findings suggest that the relationship between features and power output is highly linear and that adding further model complexity did not improve performance. This analysis highlights the importance of selecting models based on data characteristics.

1. Introduction

The problem this project aims to solve is predicting the power output of wave energy converters. To accomplish this, I've selected the "Large-scale Wave Energy Farm" dataset. This dataset was derived following a study conducted to optimize the quantity and uniform position of wave energy converters (WECs) across two wave energy farms in Australia. The dataset contains 149 columns and 36,043 rows of data. The first 98 columns contain X,Y coordinates for each of the 49 WECs. The next 49 columns labeled (Power1, Power2, ..., Power49) show the power output of each WEC at the given XY coordinate. Then, one column labeled "qW" details the q-factor, defined as "The ratio of the total power generated by the entire farm to the sum of the power outputs from all individual Wave Energy Converters (WECs) if they were operating in isolation." The final column "Total Power" contains the sum of the power generated by each WEC across all coordinates.

For this project, the key features will be the Power(n) columns, and the target variable is Total Power. The purpose of this project is to identify the machine learning model that can most effectively predict the power output. Over the next sections of this work, I will explain my exploratory data analysis to present my findings and discuss key insights. Following that, I will present my methodology from the initial data preprocessing to the evaluation metrics. I will then describe the results before ending with my conclusions and suggestions for future work.

2. Exploratory Data Analysis

The goal of the exploratory analysis was to understand the structure, quality, and relationships within the dataset to determine which features were suitable for a supervised regression task for predicting the total power output of the wave energy farm. The dataset contains 36,043 rows and 149 numerical variables, all stored as float64.

- X1-X49 and Y1-Y49: spatial coordinates of each Wave Energy Converter (WEC)
- Power1-Power49: Power generated by each WEC at corresponding coordinates
- qW: the farm-level Q-factor shown as a ratio of the total power generated by the entire farm.
- Total Power: The sum of the power generated by each WEC across all coordinates.

No missing values, invalid entries, or data quality issues were identified during the analysis.

Feature	N Features	Mean (range)	Std (range)	Min	Max
X Coordinates	49	251-755	238-433	0	1000
Y Coordinates	49	19-954	42-122	0	1010
Power(n)	49	60644-98466	3134-14998	36307	114195
Total Power	1	122617	3889	3388944	4177659

Table 1: Summary of statistics extracted from the WEC_Perth_49 dataset

Summary statistics (mean, median, standard deviation, min, and max) are shown in Table 1. This shows that all variables are continuous and vary across wide ranges. The histograms shown in Figure 1 Figure 2 displays that many power variables, including Total Power, exhibit left-skewed distributions. The X and Y coordinate values showed a broad spread with irregular distribution.

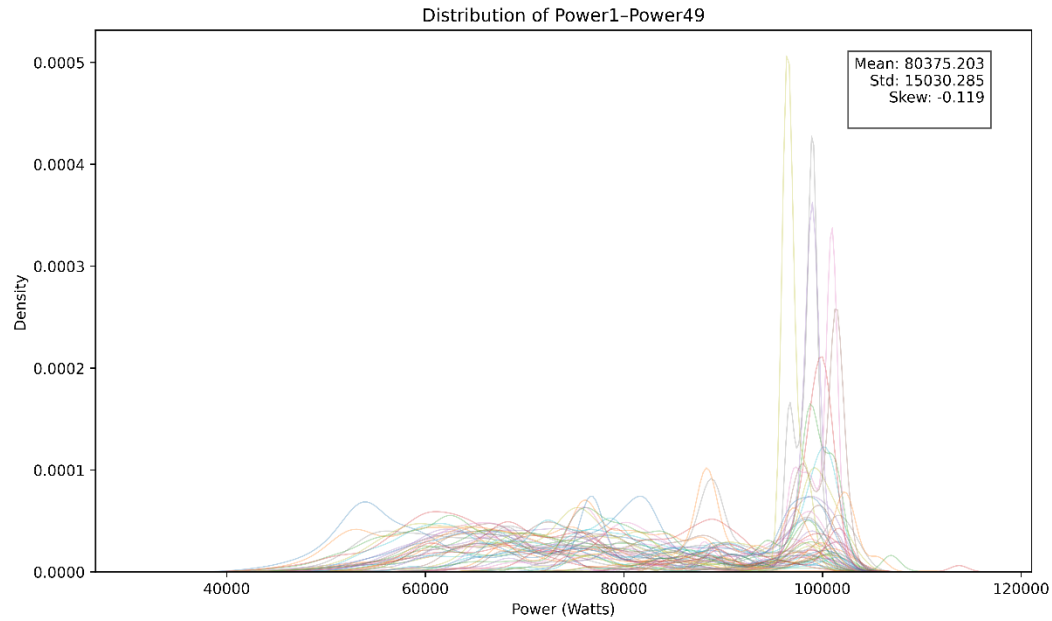


Figure 1: Distribution of all Power(n) variables with summary statistics.

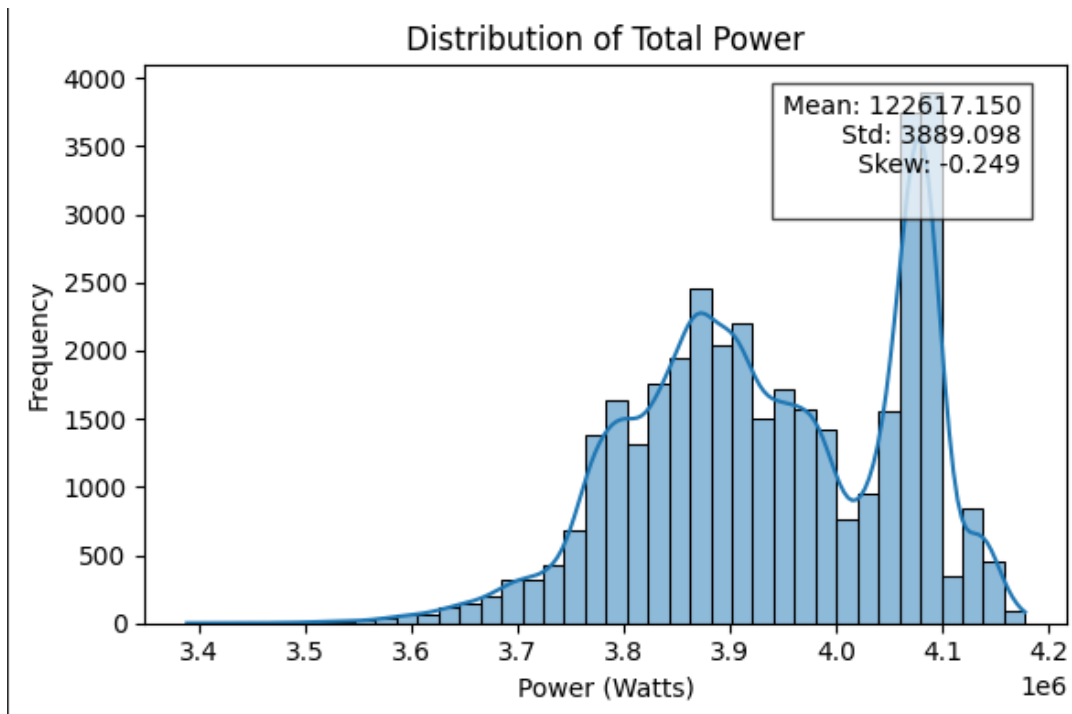


Figure 2: Distribution of Total Power with summary of statistics

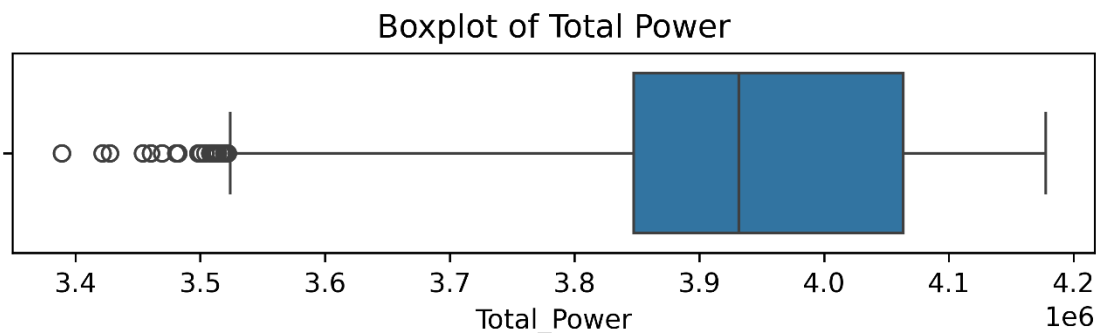


Figure 3: Boxplot of Total Power feature to demonstrate range and outliers.

Outlier detection using box plots and distribution plots found extreme values in both the Power(n) variables and Total Power (see Figure 3). However, when evaluating these values, they appear to be valid measurements rather than errors, and so they were retained.

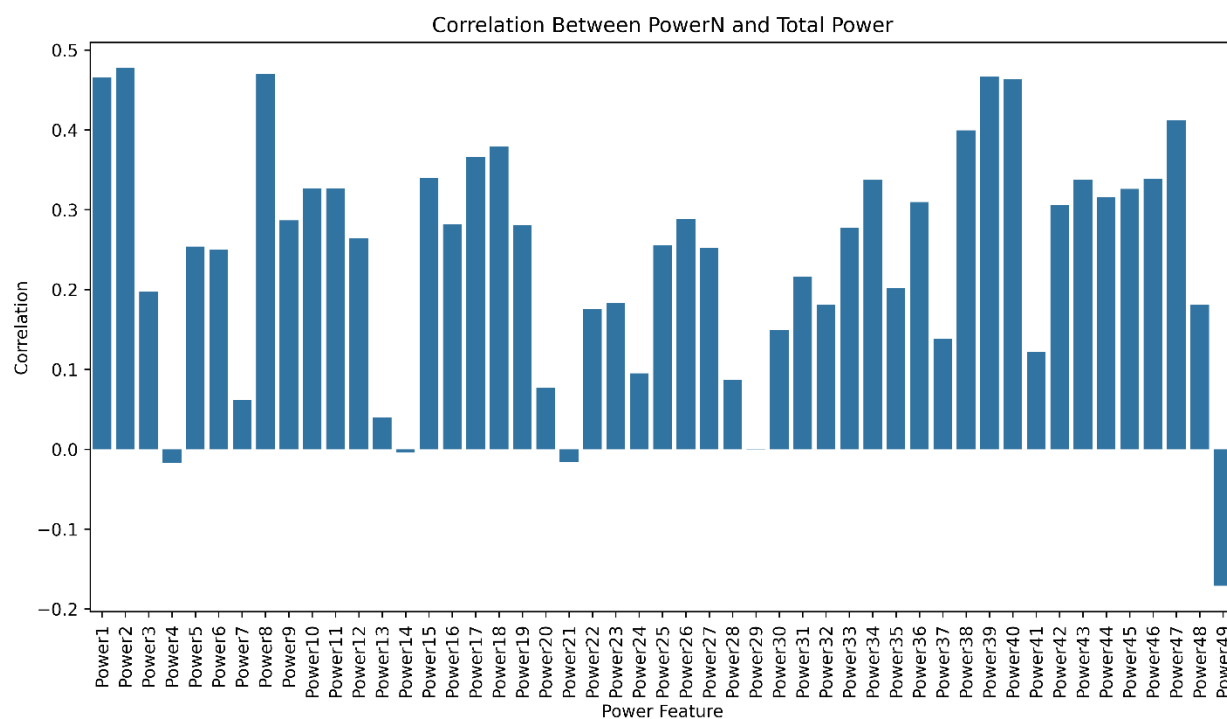


Figure 4: Correlation between each Power(n) variable and the Total Power target.

The correlation bar plot (see Figure 4) demonstrated a consistently strong positive correlation between each Power(n) variable and the Total Power target. In contrast, the X and Y coordinate features showed weak or inconsistent correlations with power output. The coordinate correlation bar plot (see Figure 5) confirmed that the coordinate data did not form predictable patterns with power output. This demonstrates that the variability in the coordinate data may be too noisy and weakly correlated to produce accurate predictions. Instead, I have opted to use the Power features as predictors of total power output, as the relationship between them is strongly supported by the correlation data gathered.

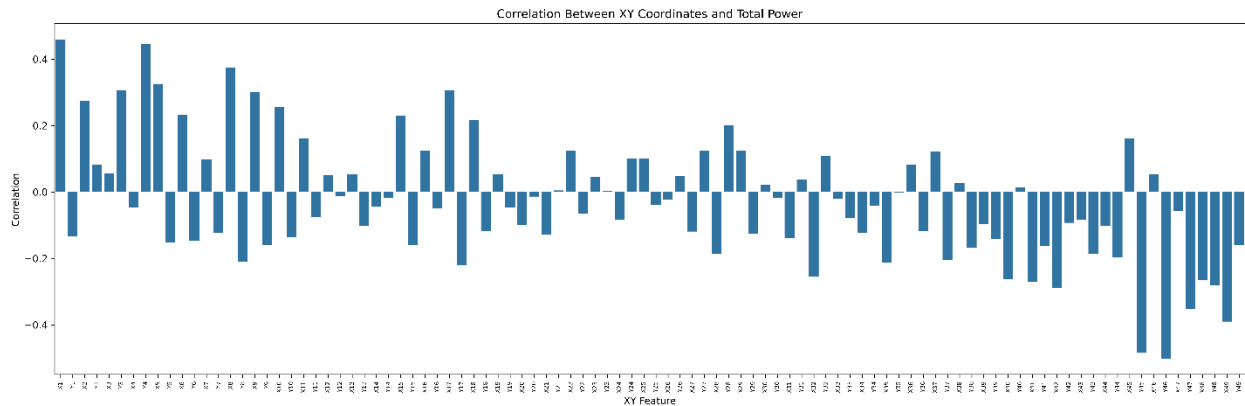


Figure 5: Correlation between each XY coordinate and the Total Power target.

The EDA phase verified that the dataset is clean, numerical, and well-structured, but not all features will be useful for prediction. The strongest relationship exists between the Power(n) variables and the Total Power output, making them the best focus for the supervised regression task.

3. Methodology

All preprocessing was performed in Python using pandas, NumPy, and scikit-learn within a Google Colab notebook. Since the dataset did not contain missing values, no imputation was required. The only feature removal was the X1-X49 and Y1-Y49 coordinates, since exploratory data analysis showed they were weakly correlated with our target variable. Removing them simplified the models and improved their generalization. All remaining numerical features were standardized using StandardScaler, which transforms each variable to zero mean and unit variance^{[1][2]}. Standardization was necessary because several models used (Ridge, Lasso, XGBoost) are sensitive to differences in feature scale.

During exploratory data analysis, it was determined that no new features would be needed. The dataset already included Total Power, which served as the target variable, and Power1-Power49, which were the predictors. Additionally, no dimensionality reduction was applied, as the data was clean, highly linear, and handled well by all models evaluated. The chosen components used to test and train the selected models are the 49 Power(n) variables due to the linearity of the variables and the strong correlation identified during EDA.

To assist in model selection, Lazy Regressor was first used to collect baseline performance and scoring across numerous models.^[3] The highest performing linear and tree-based models were then selected. The six selected models are listed below with a high-level overview of how they function.

Linear Regression (Ordinary Least Squares):

- “Fits a linear model to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximations.”^[4]

Ridge Regression (L2 Regularization)

- “Ridge adds an L2 penalty to shrink coefficients, reducing variance and mitigating multicollinearity.”^[5]

Lasso Regression (L1 Regularization)

- “Lasso applies an L1 penalty, which can shrink some coefficients to zero, performing implicit feature selection.”^{[6][7]}

Random Forest Regressor

- “A random forest is a meta estimator that fits a number of decision tree regressors on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.”^{[8][9]}

Extra Trees Regressor (Extremely Randomized Trees)

- “This class implements a meta estimator that fits a number of randomized decision trees (a.k.a. extra-trees) on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.”^{[10][11]}

XGBoost Regressor

- “A scalable gradient-boosted tree model optimized for speed and performance. Uses additive boosting to minimize a differentiable loss function.” ^{[12][13]}

For the experiment setup, the dataset was split into 80% training and 20% testing using the built-in `train_test_split` from `scikit-learn`. No separate validation set was created. Instead, all hyperparameter tuning used cross-validation inside `GridSearchCV`. ^[14]

The specific software and libraries used to conduct the exploratory data analysis, model testing, training, evaluation, scoring, and metric extraction are as follows:

- Python 3 (via Google Colab Jupyter Notebook)
- Pandas ^[15]
- Numpy ^[16]
- Lazy Regressor ^[17]
- Scikit-learn ^[18]
- XGBoost ^[12]
- Matplotlib ^[19]
- Seaborn ^[20]

All models were implemented using `scikit-learn` pipelines to ensure consistent scaling and reproducibility. ^[1]

Hyperparameter tuning was performed using `GridSearchCV` ^[21] with cross-validated negative mean squared error as the scoring metric. Negative mean squared error (NMSE) was used instead of mean squared error (MSE) because grid search tries to maximize the scoring metric. However, MSE is a loss function meaning a lower MSE indicates better model performance. To avoid conflicts with grid search’s logic, NMSE allows grid search to select the least negative MSE to maximize the score. The section below contains a list of the hyperparameters tuned and the ranges explored for each model.

Linear Regression: Achieved the highest score from Lazy Regressor, so it was used as a baseline with no hyperparameter tuning.

Ridge Regression:

- alpha: 0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000

- solver: auto, svd, cholesky, lsqr, sag, saga
- fit_intercept: True, False
- CV: 5-fold

Lasso Regression:

- alpha: 0.0001, 0.001, 0.01, 0.1, 1, 10
- fit_intercept: True, False
- selection: cyclic, random
- CV: 5-fold

Random Forest Regressor:

- n_estimators: 100
- max_depth: None, 10, 20
- min_samples_split: 2, 5
- min_samples_leaf: 1, 2, 4
- max_features: sqrt, log2
- CV: 3-fold

Extra Trees Regressor:

- n_estimators: 100
- max_depth: None, 10, 20
- min_samples_split: 2, 5
- min_samples_leaf: 1, 2, 4
- max_features: sqrt
- CV: 3-fold

XGBoost Regressor:

- n_estimators: 100
- max_depth: None, 5, 10
- learning_rate: 0.01, 0.1, 0.3
- subsample: 0.8, 1.0

- `colsample_bytree`: 0.8, 1.0
- CV: 3-fold

Fewer hyperparameter settings were tested with the tree-based models due to resource constraints. Using the standard free model of Google Colab, which provides very limited virtual compute units, the tree-based models required an unrealistic amount of time to extract the best model and parameters. As a result, the cross-validation folds were reduced from 5 to 3, and most hyperparameters were tested on their default setting plus a few others. This method still allowed for multiple variations of each tree-based model to be tested without having to wait hours for what would likely only be marginal improvements.

Once the pipeline and grid search were defined and executed for each model, the best model with its best parameters was extracted to be used in the final comparative testing. To evaluate each model fairly, four regression metrics were used to evaluate all models on both the training and test sets. These metrics are as follows: Mean Absolute Error, Mean Squared Error, Root Mean Squared Error, and Coefficient of Determination. The following section briefly describes each metric.

Mean Absolute Error (MAE): Measures average absolute deviation between prediction and true values. ^[22]

Mean Squared Error (MSE): Penalizes larger errors more heavily, making it sensitive to outliers. ^[23]

Root Mean Squared Error (RMSE): The square root of MSE and is interpretable in the same units as the target variable, in the case of this project, the units are measured in watts. ^[24]

Coefficient of Determination (R2): Measures the proportion of variance explained by the model. ^[25]

4. Results and Discussion

Rank	Model	Train MAE	Train MSE	Train RMSE	Train R2	Test MAE	Test MSE	Test RMSE	Test R2
1	Linear Regression	292.96	186884.41	432.3	0.999988	289.25	181428.84	425.94	0.999988
2	Ridge	292.83	186889.78	432.31	0.999988	289.13	181453.4	425.97	0.999988
3	Lasso	292.88	186905.51	432.33	0.999988	289.2	181507.33	426.04	0.999988
5	Random Forest	1373.96	17047650.96	4128.88	0.998872	3318.28	97264147.46	9862.26	0.993401
6	Extra Trees	1006.11	2757891.02	1660.69	0.999817	4263.33	130759262.6	11435	0.991128
4	XGBoost	828.29	2002686.29	1415.16	0.999867	2593.55	33234961.98	5764.98	0.997745

Table 2: Ranked results of each model from 1-6, evaluating on Train and Test Metrics

The performance of all six regression models was evaluated using four metrics: MAE, MSE, RMSE, and R2 across both the training and test sets, providing a comprehensive view of each model's accuracy and ability to generalize on unseen data. A complete summary of these results is presented in Table 2. The results reveal a clear performance separation between the linear and tree-based models. Linear Regression, Ridge, and Lasso consistently achieved the lowest error values and highest R2 scores across both sets, while the tree-based models: Random Forest, Extra Trees, and XGBoost lagged. Notably, the linear models displayed near-identical performance between training and test sets, suggesting strong generalization with no evidence of overfitting.

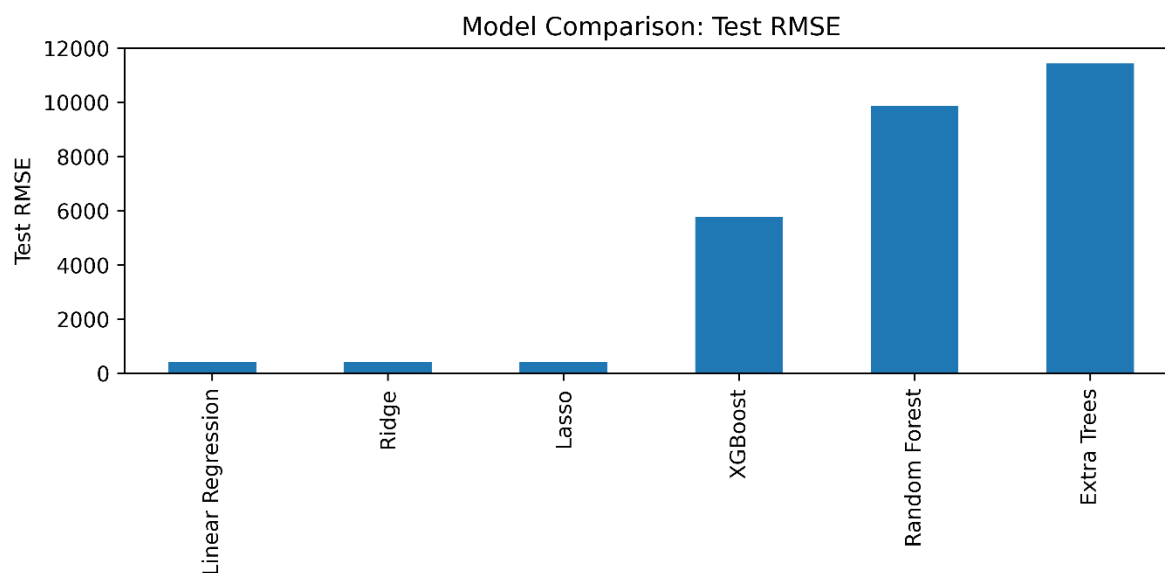


Figure 6: Model comparison by Test Root Mean Squared Error (RMSE)

To assist in demonstrating these results, several visualizations were generated. Figure 6 presents a bar chart comparing test RMSE across all models, providing a visual representation of the performance gap between linear and tree-based models. RMSE is particularly informative in this context, as it is interpreted in the same units as the target variable, making the magnitude of prediction errors directly meaningful. As observed in Table 1, Total Power is measured on the order of hundreds of thousands of watts, which makes the performance of the linear models even more respectable. Despite operating on a target of such scale, all three linear models achieved a test RMSE below 500 watts, with the best result being approximately 426 watts, achieved by the untuned Linear Regression model. This difference is a negligibly small margin of error relative to the range of the target variable.

In contrast, the tree-based models struggled to match this level of precision. XGBoost, the strongest performer among the tree-based models, recorded a test RMSE of 5,764 watts, which is roughly a 13x increase over the linear models. Random Forest and Extra Trees performed even worse, with test RMSE values of 9,862 and 11,435 watts, respectively. This gap between linear and tree-based models, combined with the larger discrepancy between training and test errors observed in the tree-based models, suggests that the relationship between the input features and Total Power is strongly linear. The added complexity of the tree-based models and hyperparameter tuning had a limited impact and instead led to overfitting to the training data and failure to generalize to unseen data.

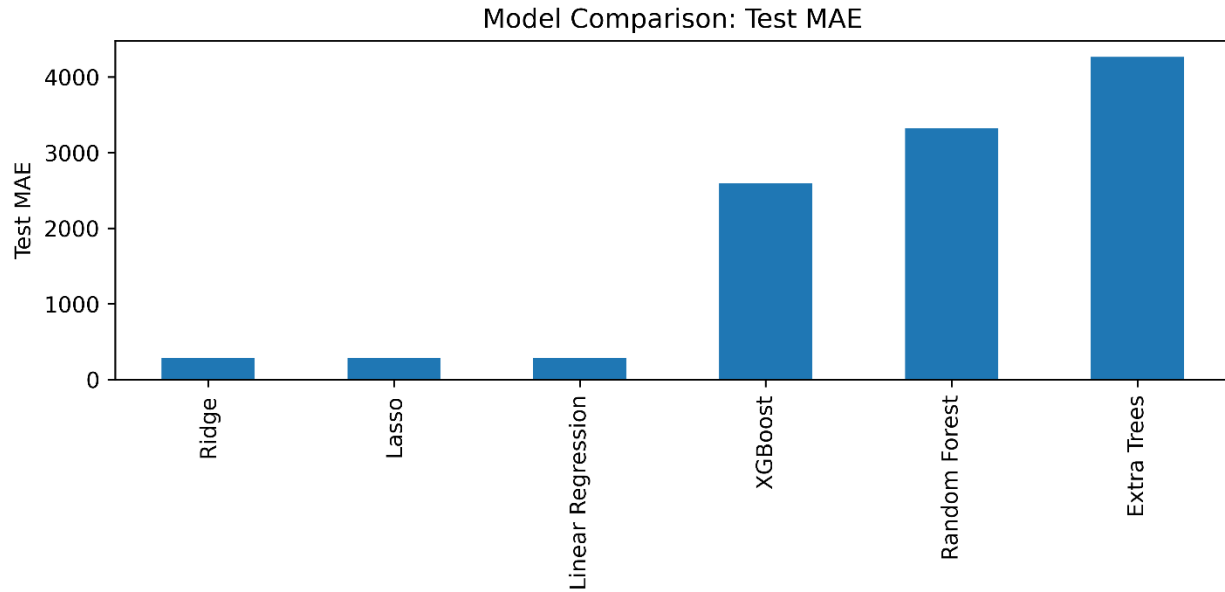


Figure 7: Model comparison by Test Mean Absolute Error (MAE)

Across all evaluation metrics, Linear Regression was the strongest performing model in this study. Its test RMSE and MAE were the lowest of all six models (see Table 2, Figure 6, Figure 7), and its R^2 was the highest, indicating that it captured nearly all of the variance present in the target variable. These results were consistent across both the training and test sets, with virtually no decrease in performance. This indicates the model is generalizing well and is not underfitting or overfitting the data.

This performance is further supported by the residual analysis presented in Figure 8, which shows the residual plot for the Linear Regression model. The residuals are broadly clustered around zero with no major curvature. However, the spread noticeably increases as Power increases, indicating the model's prediction error varies across the range of the target variable. Additionally, a visible downward curve is present at the upper end of the distribution, suggesting the model begins to underpredict at the highest power values. While these limitations are worth noting, the Linear Regression model still performed better than the other five tested. The performance of Linear Regression is consistent with the findings of the exploratory data analysis. The Power(n) features showed strong linear correlation with Total Power, suggesting the data would perform better with linear models.

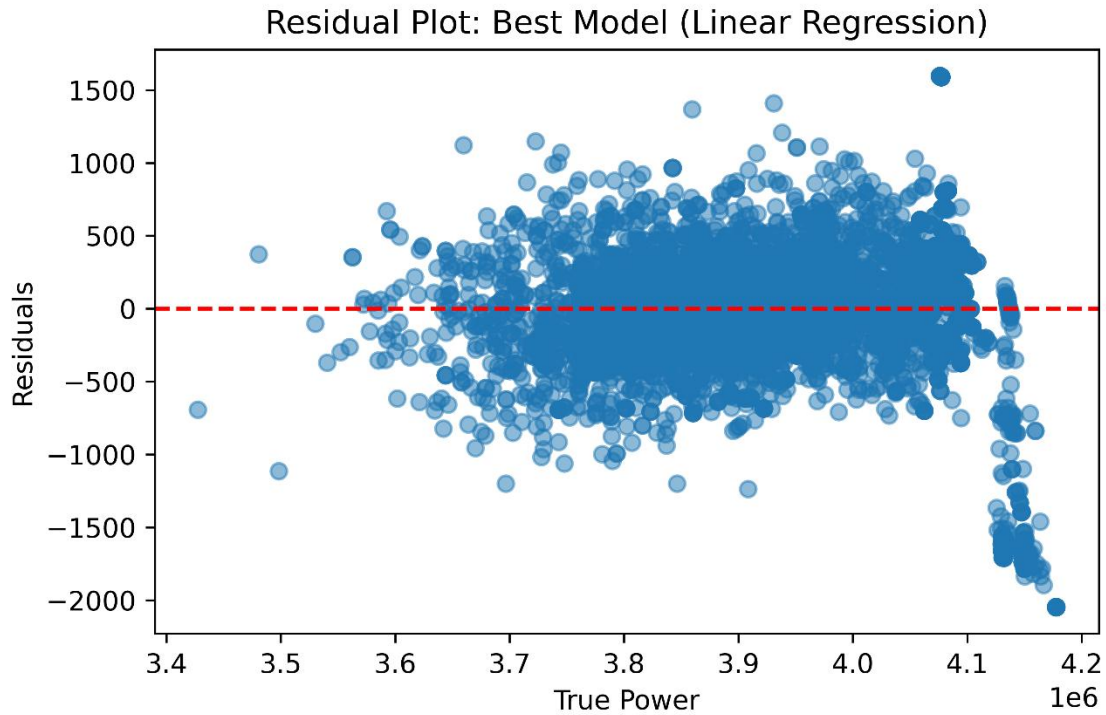


Figure 8: Residual analysis of Linear Regression

While the results of this study strongly favored the linear models, several limitations and considerations should be acknowledged. The underperformance of the tree-based models do not reflect the general capability of these models, it shows a consequence of the characteristics of this dataset. The strong linear relationship between input features and the target variable created conditions that favored simpler models. It is reasonable to assume that Random Forest, Extra Trees, and XGBoost would have performed more competitively on tasks where nonlinear relationships between features are more focused.

Additionally, the hyperparameter search conducted for the tree-based models was constrained by available resources. As noted in the hyperparameter tuning section, the experiments were conducted using the free tier of Google Colab, which caused practical limitations on the parameter grids that could be explored. A more exhaustive search with a wider range of hyperparameters may have found marginal improvements. Although, given the linearity of the data, it is unlikely that that tuning alone would have closed the gap in performance with the linear models.

5. Conclusion:

This study set out to evaluate the effectiveness of multiple regression models in predicting Total Power output from a dataset of coordinates and individual power features. Six models were assessed: Linear Regression, Ridge, Lasso, Random Forest, Extra Trees, and XGBoost, with the objective of identifying the most accurate and well-generalizing approach for this prediction task.

The key finding of this study is that Linear Regression outperformed all other models across every evaluated metric, achieving a test RMSE of approximately 426 watts against a target variable measured in the hundreds of thousands. Ridge and Lasso regression performed comparably, confirming that regularization offered no real benefit. The tree-based models failed to approach the accuracy of the linear models and showed signs of overfitting despite their greater complexity and hyperparameter tuning applied. These results suggest the relationship between the input features and Total Power is linear, and that increasing model complexity was unnecessary for the given task.

The residual analysis of Linear Regression revealed a mild fanning pattern, or heteroscedasticity, and underprediction at the upper end of the target. While these issues did not substantially impact overall model performance, they suggest the linear models precision declines at the extremes of the target range. Future work could explore whether feature engineering or target transformations such as log scaling might address these patterns and improve prediction accuracy. Additionally, future studies could investigate the performance of these models on datasets with stronger nonlinear structure, which would provide a more challenging environment for the tree-based models and offer a more complete comparison.

References

1. Pedregosa et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR.
2. Scikit-learn developers, "sklearn.preprocessing.StandardScaler," *scikit-learn*, 2024. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>.
3. "Welcome to Lazy Predict's documentation! — Lazy Predict 0.2.12 documentation," *lazypredict.readthedocs.io*. <https://lazypredict.readthedocs.io/en/latest/index.html>
4. scikit-learn developers, "sklearn.linear_model.LinearRegression — scikit-learn 0.22 documentation," *Scikit-learn.org*, 2019. https://scikitlearn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
5. "sklearn.linear_model.Ridge — scikit-learn 0.23.2 documentation," *scikit-learn.org*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html
6. "sklearn.linear_model.Lasso," *scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html
7. Tibshirani, R. (1996). *Regression Shrinkage and Selection via the Lasso*. JRSS-B.
8. scikit-learn, "3.2.4.3.2. sklearn.ensemble.RandomForestRegressor — scikit-learn 0.20.3 documentation," *Scikit-learn.org*, 2018. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
9. Breiman, L. (2001). *Random Forests*. Machine Learning.
10. "sklearn.ensemble.ExtraTreesRegressor," *scikit-learn*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.ExtraTreesRegressor.html>
11. Geurts, P., Ernst, D., & Wehenkel, L. (2006). *Extremely Randomized Trees*. Machine Learning.
12. "Introduction to Boosted Trees — xgboost 3.2.0 documentation," *Readthedocs.io*, 2025. https://xgboost.readthedocs.io/en/release_3.2.0/tutorials/model.html
13. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD.
14. Scikit-learn, "sklearn.model_selection.GridSearchCV — scikit-learn 0.22 Documentation," *Scikit-learn.org*, 2019. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
15. Pandas, "pandas documentation — pandas 1.0.1 documentation," *pandas.pydata.org*, 2024. <https://pandas.pydata.org/docs/>
16. NumPy, "Overview — NumPy v1.19 Manual," *numpy.org*, 2022. <https://numpy.org/doc/stable/>
17. "Usage — Lazy Predict 2.15 documentation," *Readthedocs.io*, 2022. <https://lazypredict.readthedocs.io/en/latest/usage.html#regression>
18. "API Reference," *scikit-learn*. <https://scikit-learn.org/stable/api/index.html>
19. "API Reference — Matplotlib 3.5.0 documentation," *matplotlib.org*. <https://matplotlib.org/stable/api/index>
20. Waskom, M. L., (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021, <https://doi.org/10.21105/joss.03021>.

21. Scikit-learn, “sklearn.model_selection.GridSearchCV — scikit-learn 0.22 Documentation,” *Scikit-learn.org*, 2019. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
22. “sklearn.metrics.mean_absolute_error,” *scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html
23. “sklearn.metrics.mean_squared_error — scikit-learn 0.24.2 documentation,” *scikit-learn.org*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html
24. “sklearn.metrics.root_mean_squared_error,” *scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.root_mean_squared_error.html
25. Scikit Learn, “sklearn.metrics.r2_score — scikit-learn 0.24.1 documentation,” *scikit-learn.org*, 2024. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html